lab1: Java concurrency

Dawa Arkhang and Zhengan Chen

September 2026

Task 1: Simple Synchronization

Task 1a: Race conditions

This code uses Runnable to implement multithreading. Threads is store in an array and use a for loop to easily create and call join() to wait for them to finish. The run() method contains a simple for loop (1_000_000 time) that increases the shared variable a by 1 each time, but a++ is not an atomic operation. When multiple thread run at the same time, they may read and write a at the same time, causing som results overwrite each other. In our case, n=4, the final printed volatile integer will be around $2M$, much lower than the actual value of $n * 1M = 4M$. Another finding is that the value of the volatile integer tends to be $1M$ when executed multiple times in a row.

Task 1b: Synchronized keyword

Here you can choose either Synchronized method or statment depending on how 1a was implemented. Since we wrote the for loop directly in run() in a), we choose Synchronized (a++) statment here. If Synchronized run() method, even if multiple threds are created, they will be executed sequentially. Once Synchronized is applied, volatile integer a is strictly equal to $n*1M$, for example, when $n = 4$, $a = 4M$.

Task 1c: Synchronization performance

run_experiments(n) is basically the original main part (at 1b) moved into a separate function, with additional code to measure the execution time. The new main method uses a for loop to run run_experiments(n) multiple times. The outer loop increases n by powers of 2, from 1 to 64. For each value of n, two additional for loops are used. The first runs run_experiments(n) X times to warm up. The second runs it Y times for the actual measurement. These Y runs are used to calculate the mean execution time, and the mean time can also be used to calculate the standard deviation.

The results show that the execution time generally increases as the number of threads increases on both the local machine and the PDC. However, the PDC

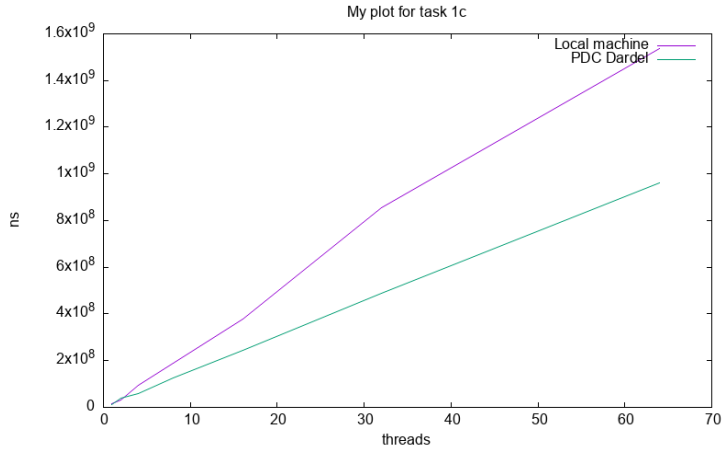

Figure 1: Task 1C: PCD vs local

has a more stable growth trend, while the local machine is less stable and the increase in execution time becomes more significant as the number of threads grows. When n is small, there is not much difference between the two. However, as n increases, the PDC performs better. Looking at the data, we can also see that on the local machine, when the number of threads doubles, the execution time increases by more than double.

Task 2

Task 2A: Implement asynchronous sender-receiver

The first part was implemented by creating two different classes that both implemented runnable, one for the incrementingThread where we are incrementing a shared counter from 0 to 1 million, and another for the printingThread which will print the current shared counter number. The result that we got after 10 test runs was that the printed number was random and was at the highest 14631 and at the lowest 3231.

Task 2B: Implement busy-waiting receiver

In comparison to part A, with the implementation of the done global variable, every single line prints sharedInt: 1000000 consistently across all 10 runs. The while (!done) loop therefore successfully synchronizes the threads so the printer never accesses sharedInt prematurely.

Task 2C: Implement a waiting with guarded block

We implemented wait() and notify() onto of the precious built code from question B by passing in the same object into the increment and printer class, connecting them. This allows us to have a notify inside the incrementer run method as soon as the 1 million iterations are done and a wait in the printer run. The wait() will allow us to suspend the thread so that it doesn't keep rerunning the same while loop checking for the done which frees up CPU power.

Task 2D: Explore the effects of guarded block on performance

The testing between the guarded block implementation and the busy waiting implementation was done by running the test a total of 30 times and only counting the 20 last runs, treating the first 10 runs as warm-up. We then take the average of the last 20 tests. The result that was found was that the busy waiting implementations had an average of 75 ns while the guarded block had a execution of 7254 ns.

Although busy-waiting exhibits lower reaction latency, it wastes 100% of an entire CPU core on empty loops, which degrades overall system throughput and starves other processes. The guarded block sacrifices several microseconds of response latency in exchange for zero CPU waste while idle, making it the proper architectural choice for scalable multi-threaded systems unless microsecond-level hardware spin-locks are required.

Task 3

Task 3A: Implement a producer-consumer buffer class with limited capacity N

We implemented a thread-safe, bounded FIFO buffer using Java's ReentrantLock alongside two explicit Condition variables: notFull and notEmpty. An ArrayDeque serves as the underlying FIFO storage, bounded by a specified capacity N. The difference between wait() notify() and the Condition is that with the usage of Condition, we are able to create two separate waiting rooms tied to the same underlying lock, allowing us to selectively wake up either a producer or a consumer. With traditional wait() and notify(), all threads share a single wait-set, which frequently causes unnecessary wake ups or forces the use of expensive notifyAll() calls. Using Condition variables (notFull and notEmpty) allows for targeted signaling, waking up only the specific thread that can make progress and significantly reducing thread contention. So when when the queue is not full anymore we only call on the notfull.signal() which will in turn only wake up the add() and not the remove(). The closed() acts as the lifecycle shutdown which tells the threads that we are done and uses signalall() on both notFull and notEmpty so no thread remain suspended indefinitely. Producers waiting

in notFull.await() wake up, observe that the buffer is closed, and abort their insertion with an IllegalStateException. Consumers waiting in notEmpty.await() wake up to safely drain any remaining items in the queue. Once the buffer is entirely empty and closed, remove() calls throw an IllegalStateException.

Task 3B: Write a program using the buffer class

Neither the consumer or the producer contains any synchronization blocks or locks as the concurrency burden is entirely encapsulated inside the Buffer class. The producer calls on the add() method 1 million times and closes the buffer right after which marks the transition of the buffer lifecycle from active to closing The consumer runs an infinite loop calling buffer.remove() without polling or sleep statements. It uses exception-driven termination, catching the IllegalStateException thrown by Buffer.remove() once the queue is fully drained.

Task 4: Counting Semaphore

Task 4a: Implement the counting semaphore

In the CountingSemaphore class, signal() and s_wait() are implemented according to the requirements. The purpose of CountingSemaphore() is to set the initial value of the semaphore to n. The s_wait() method makes a thread wait when n ¡ 1, which means there are no available resources. The signal() method wakes up one waiting thread (if it exist == semaphore ¡ 0 before semaphore++) when a resource becomes available (current thread is finish).

In Main, a CountingSemaphore object is added to Runner so that the thread can call its methods. One important detail maybe is that synchronized should be used inside the CountingSemaphore class rather than inside the Runnable class, unlike Task 1 and 2. This allows us to pass the same CountingSemaphore instance to different Runnable classes. As a result, even if the Runnable classes have different functions, they can still share and use the same semaphore.

Task 4b: Test semaphore with various counter

At first, both signal() and s_wait() used if statements for their conditions. However, considering spurious wakeups, s_wait() should use while instead of if. If a spurious wakeup occurs, a thread could continue without actually getting a resource, which could cause the number of active threads to exceed the limit n. In our tests, the if version did not cause the number of threads to exceed n, probably because no spurious wakeups occurred during the tests.

When using a semaphore, it need to call signal() when a thread finishes using the resource so that another waiting thread can continue. If a thread returns before calling signal(), the resource is not released. If this happens n times, all semaphore resources can be lost, causing waiting threads to wait forever.

Task 5

Task 5A: Model the Dining Philosophers

Task A was implemented by representing each philosopher as a Runnable object executed by a dedicated thread. To simulate the table, an array of shared Object instances acted as the chopsticks. We passed references to the corresponding left and right chopstick objects into each philosopher's constructor, ensuring that adjacent threads competed for the exact same resources and not for some local one.

We thereafter simply created a nested synchronized block where the outer block locked the left chopstick and the inner block locked the right chopstick. The eating action itself was simulated by calling Thread.sleep() within the critical section.

Initially, the simulation suffered from immediate deadlock which lead to us frequently recording a deadlock time of 0 ms. Because all threads were spawned simultaneously and executed at maximum speed, they instantaneously acquired their left chopsticks and blocked on the right, creating a circular wait before any meaningful execution could occur.

To overcome this issue and allow the system to run dynamically, we introduced a randomized "thinking" period prior to resource acquisition. This delay staggered the threads, breaking the initial lockstep synchronization and allowing philosophers to successfully eat for a variable duration before naturally stumbling into a genuine deadlock.

Number of Philosophers (N)	**Avarage time to Deadlock (ns)**	**Time to Deadlock (ms)**
5	27,227,193	≈ 27.2
10	64,945,870	≈ 64.9
15	216,397,250	≈ 216.4

Table 1: Observation of time until deadlock occurs for varying numbers of philosophers.

Task 5B: What debugging tools does the Java environment offer that might help us debug this deadlock?

The tool that we used to analyze the deadlock was jps and jstack which allows the user to find the running processes IDs through jps and then parsing that found ID to jstack. This makes the JVM generates a complete thread dump with an automated deadlock analyzer. When applied to our frozen simulation (with $N = 15$), jstack immediately flagged a system failure, reporting: Found

1 deadlock.The resulting thread stack dump explicitly mapped out the circular wait.

Thread-0 was shown holding monitor lock 0x000000070fe1a560 (its left chopstick) while blocked waiting for monitor 0x000000070fe1a570 (its right chopstick), which was held by Thread-1. This chain continued unbroken all the way through to Thread-14, which looped back to block on Thread-0's lock.

Task 5C: Implement a solution to the Dining Philosophers problem

The core idea of the solution is that a philosopher only picks up the left and right chopsticks when both are available. Checking and picking up the chopsticks are treated as one atomic operation, canPick(). The reason why both checking and picking up need to be atomic is to avoid the situation of having chopsticks at the same time. This is achieved via ReentrantLock, where the philosopher needs to acquire a lock. If one of the chopsticks has already been used, the Philosopher enters the wait state and releases the ReentrantLock via condition.await().

Each thread (with new Runnable) represents a philosopher, and each thread has an ID (start from 0) that is used as its position index. The chopsticks are represented by a boolean array, where true means the chopstick is currently being used. Each philosopher can only pick up chopstick id and chopstick $(id + 1)\%n$. Waiting is implemented using an array of Condition. Each thread has its own Condition based on its ID. Each thread repeatedly calls eat(), and after canPick() succeeds, it sleeps for a random amount of time to simulate eating. After eating, the philosopher puts down the chopsticks and wakes up any neighboring philosophers who may be waiting.

The deadlock-free logic is to prevent a philosopher from holding one chopstick while waiting for the other. If both chopsticks cannot be picked up at the same time, the philosopher picks up neither. However, this does not necessarily guarantee starvation freedom. A simple, although potentially inefficient, solution is to use a queue. A philosopher must join the queue before trying to eat. Therefore, a philosopher can eat only if both chopsticks are available and the philosopher is the first one in the queue. After finishing eating, the philosopher signals all waiting philosophers instead of only the two neighbors.